



JPA CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & PERSISTENCE.XML

Installation

```
<persistence-unit name="myPU" transaction-typ...
<class>com.example.User</class>
<properties>..db props..</properties>
</persistence-unit>
@PersistenceUnit EntityManagerFactory emf;
```

04. JPQL QUERIES

Workflow

```
TypedQuery<User> q = em.createQuery(
"SELECT u FROM User u WHERE u.active = :a", U...
q.setParameter("a", true);
List<User> list = q.getResultList();
User u = q.getSingleResult();
```

JPQL uses entity/field names, not SQL table/column names

02. ENTITY MANAGER

Architecture

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
em.persist(entity); // INSERT
em.merge(entity); // UPDATE detached
em.remove(entity); // DELETE
em.find(User.class, id); // SELECT by PK
em.getTransaction().commit();
```

05. NAMED QUERIES

CRUD API

```
@NamedQuery(name="User.findByEmail",
query="SELECT u FROM User u WHERE u.email=:e")
em.createNamedQuery("User.findByEmail", User....
.setParameter("e", email).getSingleResult();
```

Parsed at startup fail fast on typos

03. ENTITY MAPPING

YAML / Config

```
@Entity @Table(name="users")
public class User implements Serializable {
@Id @GeneratedValue(strategy=GenerationType.I...
private Long id;
@Column(nullable=false, length=100)
private String name;
}
```

06. RELATIONSHIPS

Integration

```
@OneToOne @OneToMany @ManyToOne @ManyToMany
@JoinColumn(name="user_id") FK column
@JoinTable for ManyToMany bridge table
```

mappedBy on non-owning (inverse) side
Bidirectional: manage both sides in code



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. FETCH TYPES

Hardening

LAZY (default for *ToMany): loads on first access
EAGER (default for *ToOne): loads with parent

```
@OneToMany(fetch = FetchType.EAGER) // N+1 ri...
```

Fix N+1: use JOIN FETCH in JPQL

```
"FROM Order o JOIN FETCH o.items"  
@EntityGraph for ad-hoc fetch plans
```

10. TRANSACTIONS

Unit Tests

```
@Transactional // Spring integration  
em.getTransaction().begin() // resource-local  
@PersistenceContext EntityManager em // inje...
```

JTA: container-managed in Jakarta EE

```
@Transactional(propagation=REQUIRES_NEW)
```

08. CASCADE & ORPHAN REMOVAL

Observability

```
@OneToMany(cascade=CascadeType.ALL, orphanRem...
```

CascadeType: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL
orphanRemoval=true auto-deletes children removed from list
Cascade REMOVE on Many side = delete all children
Only cascade from parent to child, not vice versa

11. CRITERIA API

Commands

```
CriteriaBuilder cb = em.getCriteriaBuilder();  
CriteriaQuery<User> cq = cb.createQuery(User...  
Root<User> r = cq.from(User.class);  
cq.where(cb.equal(r.get("status"), "active"));  
em.createQuery(cq).getResultList();
```

Type-safe; use metamodel (User_.name) for extra safety

09. ENTITY LIFECYCLE CALLBACKS

Optimization

```
@PrePersist @PostPersist  
@PreUpdate @PostUpdate  
@PreRemove @PostRemove  
@PostLoad  
@EntityListeners(AuditListener.class)
```

Used for auditing: createdAt, updatedAt auto-fill

12. COMMON GOTCHAS

Warning

Detached entities: merge() instead of persist()
LazyInitializationException outside transaction scope
Bidirectional always sync both sides
GenerationType.AUTO may use SEQUENCE on PostgreSQL
@Transactional only works on Spring-managed beans